

Choosing the Learning Rate (α)

Selecting a good learning rate is critical for the performance of gradient descent. An incorrect choice can lead to slow training or a complete failure to converge.

How to Tell if Your Learning Rate is Incorrect

You can diagnose issues with your learning rate by examining the **learning curve** (J vs. number of iterations).

- **If α is too large:**
 - The cost J might **increase and decrease** (oscillate) over iterations. This happens when the algorithm repeatedly **overshoots** the minimum.
 - In a more extreme case, the cost J might **consistently increase** on every iteration. This is a clear sign of **divergence**.
 - **Solution:** In both cases, the fix is to **try a smaller learning rate α** .
 - **If α is too small:**
 - The cost J will decrease consistently, but **very slowly**.
 - **Solution:** While this will eventually work, it's inefficient. You should try a larger value for α to speed up convergence.
-

A Method for Choosing α

A practical approach is to test a range of α values and observe which one performs best.

1. **Start with a range of values:** It's common to try values on a logarithmic scale, often increasing by a factor of approximately 3 each time.
 - **Example values to try:** 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, etc.
 2. **Test and Observe:** For each value of α , run gradient descent for a moderate number of iterations and plot the learning curve.
 3. **Analyze the Curves:**
 - Identify values that are **too large** (cost diverges or oscillates).
 - Identify values that are **too small** (cost decreases too slowly).
 4. **Select the Best α :** Choose the learning rate that causes the cost to **decrease quickly and consistently**. Often, the best choice is the largest value that works without diverging, or one slightly smaller than that.
-

Debugging Tip 🐛

If your cost J is not decreasing on every iteration, it's important to distinguish between a bad learning rate and a bug in your code.

- **Test:** Set α to a **very small number** (e.g., 0.0001).
- **Result:**
 - If J now decreases on every iteration (even if slowly), your original α was too large.
 - If J *still* increases or oscillates, you likely have a **bug in your code** (e.g., using + instead of - in the update rule).